CENTRO UNIVERSITÁRIO DO DISTRITO FEDERAL – UDF

COORDENAÇÃO DO CURSO DE SISTEMAS DE INFORMAÇÃO

Anna Beatriz Resende Oki Corrêa

Protótipo para converter fórmulas de planilha com condicionais aninhadas em Python

BRASÍLIA

2026

Anna Beatriz Resende Oki Corrêa

Protótipo para converter fórmulas de planilha com condicionais aninhadas em Python

Trabalho de conclusão de curso apresentado à Coordenação do Curso de Sistemas de Informação, do Centro Universitário do Distrito Federal - UDF, como requisito parcial para obtenção do grau de bacharel / tecnólogo em Sistemas de Informação.

Orientador: Prof. Dr. Xxxx

BRASÍLIA

?Sobrenome?, ?Nome?. Título : ?author?. – Brasília, 2026. 21 f.

Orientador: ?orientador?. Trabalho de Conclusão de Curso (Graduação – Sistemas de Informação) – Centro Universitário do Distrito Federal – UDF. Coordenação do Curso de Sistemas de Informação, Brasília, DF, 2026.

1. ?Assunto?. 2. ?Assunto?. 3. ?Assunto?. I. ?Título?.

CDU: ?consultar na biblioteca?

?Titulo?

Trabalho de conclusão de curso apresentado à ?Coordenação?, do Centro Universitário do Distrito Federal – UDF, como requisito parcial para obtenção do grau de bacharel / tecnólogo em ?Curso?.

Orientador: ?orientador?.

Brasília, _____ de _____ de 2026.

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

Nota: _____

Dedico este trabalho à minha família, pelo apoio, incentivo e compreensão durante toda a minha trajetória acadêmica.

Agradecimentos

Agradeço, primeiramente, a Deus, por mais uma conquista; ao meu orientador, pela dedicação e correções; e aos bibliotecários pelo suporte em todas as pesquisas.

"A educação é a arma mais poderosa que você pode usar
para mudar o mundo."

Nelson Mandela

Resumo

Profissionais de tecnologia precisam, com frequência, passar uma mesma regra de uma representação para outra, como de uma fórmula de planilha para um código. Quando a fórmula tem condições aninhadas, a lógica fica escondida e é fácil esquecer uma condição. Este trabalho propõe um protótipo que converte fórmulas de planilha com condições aninhadas em código Python e mostra a lógica em forma de fluxograma. A lógica da fórmula é organizada em uma árvore de condições, da qual saem o código e o fluxograma. A avaliação compara o resultado da fórmula com o do código em casos de teste e reúne a opinião de um profissional sobre a utilidade do fluxograma. O trabalho está em andamento: o problema, a proposta e o plano de avaliação estão definidos, e o desenvolvimento do protótipo será a próxima etapa.

Palavras-chave: *lógica computacional; fórmulas de planilha; conversão de código; fluxograma; Python.*

Abstract

Technology professionals often need to move the same rule from one representation to another, such as from a spreadsheet formula to code. When a formula has nested conditions, the logic is hidden and it is easy to miss a condition. This work proposes a prototype that converts spreadsheet formulas with nested conditions into Python code and shows the logic as a flowchart. The formula's logic is organized as a tree of conditions, from which the code and the flowchart are generated. The evaluation compares the formula's result with the code's result on test cases and collects a professional's opinion on the usefulness of the flowchart. The work is in progress: the problem, the proposal and the evaluation plan are defined, and the prototype development is the next step.

Keywords: *computational logic; spreadsheet formulas; code conversion; flowchart; Python.*

Lista de ilustrações

1 Relação entre diferentes formas de representação da lógica computacional. . . . 15

Sumário

1 INTRODUÇÃO 11

2 FUNDAMENTAÇÃO TEÓRICA 14
2.1 Abstração e lógica computacional 14
2.1.1 Abstração 14
2.1.2 Lógica independente da forma 14
2.1.3 Formas de representar a lógica 14
2.2 Fluxogramas 15
2.3 Fórmulas de planilha e condições aninhadas 16
2.4 Conversão de código e verificação 16

3 PROPOSTA DO PROTÓTIPO 17
3.1 O problema 17
3.2 Visão geral da solução 17
3.3 Como o protótipo funciona 17
3.4 Escopo 18
3.5 Plano de avaliação 18
3.6 Limitações 19
3.7 Situação atual 19

4 CONSIDERAÇÕES FINAIS 20

REFERÊNCIAS 21

1 Introdução

A lógica computacional é a base do desenvolvimento de software. Ela define as regras, as decisões e as condições que fazem um sistema se comportar de determinada forma. Para ser usada, essa lógica precisa ser escrita de algum jeito que o computador consiga executar.

A abstração ajuda a lidar com a complexidade da computação. Com ela, deixam-se de lado os detalhes de uma implementação para olhar só o que importa em um problema (ABELSON; SUSSMAN, 1996). Por isso, uma mesma lógica pode ser escrita de formas diferentes. Uma condição, por exemplo, pode aparecer como um `if` em uma linguagem de programação, como um losango em um fluxograma ou como um nó em um sistema visual. A forma muda, mas o comportamento esperado continua o mesmo. Neste trabalho, chama-se de **lógica** o conjunto de regras e decisões, e de **representação** a forma usada para escrevê-las. A programação visual, que usa elementos gráficos para construir programas, é um exemplo de representação (MYERS, 1990; NOONE; MOONEY, 2018).

Na prática, profissionais de tecnologia precisam passar uma mesma lógica de uma representação para outra. Isso acontece, por exemplo, quando uma regra escrita em uma fórmula de planilha precisa virar um código, ou o contrário. Nessa passagem, a lógica não pode mudar. Se uma condição for esquecida, o resultado fica errado.

Esse problema apareceu em um questionário exploratório aplicado a quatro profissionais de Tecnologia da Informação. Um deles, que usa programação como apoio técnico no trabalho, relatou que precisa levar fórmulas de planilha para Python, e também o caminho inverso, mantendo o mesmo resultado. A maior dificuldade são as fórmulas com condições aninhadas, isto é, um `SE` dentro de outro `SE`. Nelas, a lógica fica escondida na sintaxe e é fácil deixar passar uma condição. Hoje ele faz esse trabalho à mão: lista as condições, reescreve a solução e testa. Segundo ele, isso gera retrabalho, e a perda de tempo é a consequência mais importante. Dois outros participantes relataram dificuldades parecidas ao adaptar uma lógica entre tecnologias diferentes. Como foram só quatro respostas, o questionário mostra que a necessidade existe, mas não permite dizer que todos os profissionais passam por isso.

A literatura mostra que passar código de uma linguagem para outra é uma tarefa difícil, mesmo para modelos de Inteligência Artificial. Pan et al. (2024) avaliaram a tradução de código entre cinco linguagens e consideraram uma tradução correta apenas quando o código compila, executa e passa nos testes. Nesse estudo, o percentual de traduções corretas ficou entre 2,1% e 47,3%, conforme o modelo. Isso mostra que gerar um código que parece certo não basta: é preciso verificar se ele se comporta como o original. Nos trabalhos consultados até agora, não foi encontrado

um estudo que trate especificamente da conversão de fórmulas de planilha com condições aninhadas para código.

Diante disso, o **problema** deste trabalho é a dificuldade de converter fórmulas de planilha com condições aninhadas em código sem perder nenhuma condição, e de conferir se o resultado continua igual.

O **objetivo geral** é desenvolver e avaliar um protótipo que converte fórmulas de planilha com condições aninhadas em código Python e mostra a lógica em forma de fluxograma. Os **objetivos específicos** são:

- definir quais tipos de fórmula o protótipo vai aceitar;
- representar a lógica da fórmula em uma estrutura independente da forma em que ela foi escrita;
- gerar, a partir dessa estrutura, o código Python e o fluxograma;
- verificar se o código gerado dá o mesmo resultado da fórmula em casos de teste;
- avaliar, com um profissional da área, se o fluxograma ajuda a entender a lógica.

A **solução proposta** é uma ferramenta pequena. Ela recebe uma fórmula, identifica suas condições e organiza essa lógica em uma estrutura em forma de árvore. A partir dessa árvore, a ferramenta gera o código Python e o fluxograma. Como os dois saem da mesma estrutura, a lógica é a mesma nas três formas: fórmula, código e fluxograma. Essa estrutura faz o papel de representação intermediária, ideia que vem dos estudos sobre abstração e representação da lógica. O uso de regras fixas ou de Inteligência Artificial para fazer a conversão será definido durante o desenvolvimento.

O público a que se destina são profissionais que usam planilhas e códigos no trabalho e precisam passar regras de uma forma para a outra. O profissional que relatou o problema será o principal usuário na avaliação.

Do ponto de vista metodológico, este trabalho é uma pesquisa aplicada, com desenvolvimento de um protótipo (GIL, 2002; PRODANOV; FREITAS, 2013). As etapas são: pesquisa bibliográfica, levantamento de fórmulas reais e de exemplo, desenvolvimento do protótipo e avaliação. A avaliação terá duas partes: testes que comparam o resultado da fórmula com o do código gerado, e a opinião do profissional sobre o fluxograma. Ficam fora do escopo outras linguagens, programação por blocos e uma aplicação web completa.

Este trabalho está dividido em quatro capítulos. Esta introdução apresenta o contexto, o problema, os objetivos e a metodologia. O Capítulo 2 apresenta a base teórica sobre abstração,

representações da lógica e programação visual. O Capítulo 3 apresenta a proposta do protótipo e o plano de avaliação. O Capítulo 4 traz as considerações finais.

2 Fundamentação Teórica

2.1 ABSTRAÇÃO E LÓGICA COMPUTACIONAL

2.1.1 Abstração

A abstração é uma ideia central da computação. Com ela, um sistema complexo é descrito de forma simples, mostrando só o que importa para um determinado fim. Segundo Abelson e Sussman (1996), a abstração permite trabalhar um problema em diferentes níveis, sem precisar lidar com todos os detalhes de como ele é implementado.

No desenvolvimento de software, a abstração aparece em várias camadas, das linguagens de programação até as telas que o usuário vê. Cada camada esconde parte da complexidade da camada de baixo (ABELSON; SUSSMAN, 1996).

2.1.2 Lógica independente da forma

A lógica computacional é o conjunto de regras, decisões e passos que definem o que um sistema faz. Ela pode ser escrita de várias formas, e a forma não muda o que a lógica faz. Knuth (1968) mostra que um algoritmo existe independentemente da linguagem em que é escrito. A mesma solução pode ser escrita em linguagens diferentes, e o que muda é só a forma.

Uma decisão simples, como *se a idade for maior ou igual a 18, é maior de idade, senão é menor*, pode ser escrita como um `if` em Python, como uma fórmula `SE` em uma planilha ou como um losango em um fluxograma. As três formas parecem diferentes, mas têm a mesma lógica.

Neste trabalho, **lógica** é o conjunto de regras e decisões, e **representação** é a forma usada para escrevê-las. Essa distinção é a base da proposta: se a lógica é separada da forma, é possível guardá-la de modo neutro e escrevê-la de novo em outra forma.

2.1.3 Formas de representar a lógica

As representações da lógica podem ser textuais ou visuais. As textuais incluem as linguagens de programação, o pseudocódigo e as fórmulas de planilha. As visuais usam desenhos, como os fluxogramas. Algumas existem só para descrever e comunicar um algoritmo, e outras

podem ser executadas.

Myers (1990) organizou as formas de programação visual e de visualização de programas. A visualização de programas, que mostra visualmente um programa já existente, é diferente da programação visual, em que o programa é construído com elementos gráficos. Este trabalho usa o fluxograma como visualização: ele mostra a lógica de uma fórmula, mas não é onde o programa é construído.

2.2 FLUXOGRAMAS

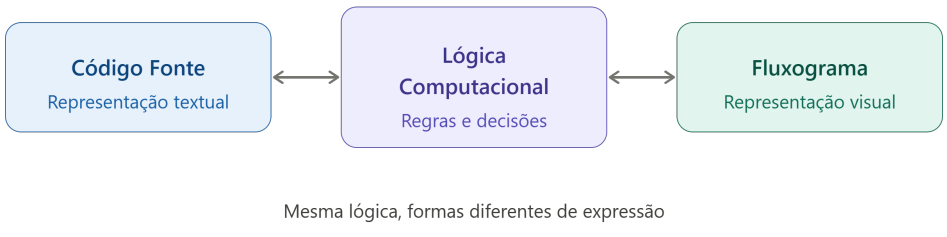
Os fluxogramas são uma das formas mais antigas de representar algoritmos de modo visual. Com símbolos padronizados e setas, eles mostram decisões, passos e caminhos de execução. Por isso, são muito usados para documentar e explicar uma lógica.

Um fluxograma é descritivo: ele mostra a lógica, mas não a executa. Para rodar, a lógica precisa ser escrita em código. Essa característica combina com o objetivo deste trabalho, que é usar o fluxograma para ajudar a entender e conferir a lógica de uma fórmula, e não para executá-la.

Myers (1990) diferencia a programação visual, em que o programa é construído com elementos gráficos, da visualização de programas, em que um programa já existente é mostrado de forma visual. Na literatura, a relação entre representações visuais e textuais é estudada principalmente no ensino de programação (NOONE; MOONEY, 2018). Neste trabalho, o fluxograma é usado como visualização de uma lógica que já existe.

A Figura 1 mostra a ideia: o código e o fluxograma são duas formas de expressar a mesma lógica.

Figura 1 – Relação entre diferentes formas de representação da lógica computacional.



Fonte: elaboração do autor com base em Abelson e Sussman (1996) e Knuth (1968).

2.3 FÓRMULAS DE PLANILHA E CONDIÇÕES ANINHADAS

Em planilhas, as regras são escritas como fórmulas. A função `SE` recebe uma condição e dois resultados: o que vale se a condição for verdadeira e o que vale se for falsa. Quando existem mais de duas situações, um `SE` é colocado dentro de outro. É o que se chama de condições aninhadas. Por exemplo:

```
=SE(A1>10; "alto"; SE(A1>5; "médio"; "baixo"))
```

Essa fórmula tem três resultados possíveis. Com mais níveis, ela fica longa e difícil de ler, porque as condições ficam escondidas dentro dos parênteses. Esse é o ponto em que é fácil esquecer uma condição ao passar a regra para outro formato, como foi relatado no questionário apresentado na introdução.

Essa mesma lógica pode ser organizada como uma árvore: cada decisão é um nó, e cada caminho termina em um resultado. A árvore não depende da sintaxe da planilha nem da do Python, e é a partir dela que a proposta deste trabalho gera o código e o fluxograma.

2.4 CONVERSÃO DE CÓDIGO E VERIFICAÇÃO

Passar uma lógica de uma forma para outra exige que o comportamento se mantenha. Em estudos de tradução de código entre linguagens, uma tradução só é considerada correta quando o código compila, executa e passa nos testes (PAN et al., 2024). Código que apenas parece correto não é suficiente.

Por isso, a verificação deste trabalho é feita com casos de teste. Os mesmos valores de entrada são aplicados à fórmula e ao código gerado, e os resultados são comparados. Se forem iguais em todos os casos, o código é considerado equivalente à fórmula para aqueles casos. Os testes mostram que o código funciona nos casos escolhidos, mas não provam que ele está correto em todas as situações possíveis. Por isso, a escolha dos casos é parte importante da avaliação.

3 Proposta do Protótipo

3.1 O PROBLEMA

Em planilhas, é comum escrever regras de negócio por meio de fórmulas. Quando a regra tem várias situações possíveis, a fórmula passa a ter condições dentro de condições. Essas fórmulas são difíceis de ler, porque a lógica fica escondida na sintaxe. Ao levar a regra para um código, como o Python, é fácil esquecer uma das condições.

Hoje, esse trabalho é feito à mão: a pessoa lista as condições, reescreve a solução e testa alguns casos. Isso toma tempo e depende de atenção. Esse relato vem do questionário exploratório descrito na introdução e, por isso, indica uma necessidade real, sem permitir generalizações.

3.2 VISÃO GERAL DA SOLUÇÃO

A solução proposta é um protótipo que recebe uma fórmula de planilha com condições aninhadas e entrega três coisas:

- o código Python equivalente;
- um fluxograma com a lógica da fórmula;
- um relatório de testes que compara o resultado da fórmula com o do código.

O principal usuário é o profissional que precisa passar regras entre planilha e código. O protótipo deve ser simples: não é preciso uma aplicação web completa, e basta que ele seja usado em casos reais de pequeno porte.

3.3 COMO O PROTÓTIPO FUNCIONA

O funcionamento tem quatro etapas:

1. **Leitura:** a fórmula é lida e suas partes são identificadas (condições, valores e resultados).
2. **Árvore de condições:** a lógica é organizada em uma árvore, em que cada nó é uma decisão e cada caminho leva a um resultado.

3. **Geração:** da mesma árvore saem o código Python e o fluxograma.
4. **Verificação:** os mesmos casos de teste são aplicados à fórmula e ao código, e os resultados são comparados.

A árvore de condições é o centro da proposta. Ela é a representação intermediária: não depende da sintaxe da planilha nem da do Python. Por isso, os dois resultados saem da mesma lógica, o que reduz o risco de uma condição aparecer em um e faltar no outro. Essa escolha segue a ideia de abstração discutida no Capítulo 2 (ABELSON; SUSSMAN, 1996).

3.4 ESCOPO

Na primeira versão, o protótipo deve aceitar fórmulas com:

- condições do tipo `SE`, aninhadas;
- comparações simples (maior, menor, igual);
- combinações com `E` e `OU`;
- resultados que sejam números ou textos.

Ficam fora do escopo: outras linguagens de destino, programação por blocos, *blueprints* e uma aplicação web completa. O uso de regras fixas ou de Inteligência Artificial para a conversão será decidido durante o desenvolvimento, conforme a viabilidade e os resultados dos testes.

3.5 PLANO DE AVALIAÇÃO

A avaliação terá duas partes.

1. **Correção:** fórmulas reais, fornecidas pelo profissional (sem dados sigilosos), e fórmulas de exemplo criadas pela autora serão convertidas. Para cada uma, será montado um conjunto de casos de teste. O código é considerado correto quando dá o mesmo resultado da fórmula em todos os casos. Gerar um código que apenas roda sem erro não é suficiente.
2. **Utilidade do fluxograma:** o profissional que relatou o problema vai analisar os fluxogramas e dizer se eles ajudam a entender a lógica e a encontrar condições esquecidas.

Também serão registrados os erros e as fórmulas que o protótipo não consegue converter, para discutir seus limites.

3.6 LIMITAÇÕES

O trabalho se limita a um tipo de problema e a um usuário principal, e os resultados não podem ser generalizados para todos os profissionais. A quantidade de fórmulas avaliadas será pequena. Além disso, o protótipo não cobre todas as funções de planilha. Essas limitações serão discutidas junto com os resultados.

3.7 SITUAÇÃO ATUAL

Até o momento, o problema, a solução e o plano de avaliação foram definidos. O protótipo, o levantamento das fórmulas reais e a avaliação ainda serão realizados.

4 Considerações Finais

Este trabalho parte de um problema concreto: converter fórmulas de planilha com condições aninhadas em código sem perder nenhuma condição. Esse problema foi relatado por um profissional de tecnologia em um questionário exploratório.

Para tratar esse problema, foi proposto um protótipo que organiza a lógica da fórmula em uma árvore de condições e, a partir dela, gera o código Python e um fluxograma. Essa proposta usa o que foi estudado sobre abstração e representação da lógica: a mesma lógica pode ser escrita de formas diferentes sem mudar o comportamento.

Até aqui, foram definidos o problema, os objetivos, o escopo e o plano de avaliação. O protótipo ainda não foi desenvolvido, e por isso ainda não há resultados.

Os próximos passos são: reunir fórmulas reais e de exemplo, desenvolver o protótipo, aplicar os testes de correção e coletar a opinião do profissional sobre os fluxogramas. Com esses resultados, será possível discutir o que funcionou, quais fórmulas o protótipo não consegue converter e que melhorias seriam necessárias.

Referências

ABELSON, H.; SUSSMAN, G. J. Structure and Interpretation of Computer Programs. [S.l.]: MIT Press, 1996.

GIL, A. C. Como elaborar projetos de pesquisa. 4. ed. São Paulo: Atlas, 2002.

KNUTH, D. E. The Art of Computer Programming. [S.l.]: Addison-Wesley, 1968. v. 1.

MYERS, B. A. Taxonomies of visual programming and program visualization. Journal of Visual Languages and Computing, v. 1, n. 1, p. 97–123, 1990.

NOONE, M.; MOONEY, A. Visual and textual programming languages: A systematic review of the literature. Journal of Computers in Education, v. 5, n. 2, p. 149–174, 2018.

PAN, R. et al. Lost in translation: A study of bugs introduced by large language models while translating code. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE '24). Lisboa, Portugal: [s.n.], 2024.

PRODANOV, C. C.; FREITAS, E. C. d. Metodologia do trabalho científico. 2. ed. Novo Hamburgo: Feevale, 2013.